
PRODUCT REQUIREMENTS DOCUMENT (PRD)

Project Title

Adaptive Noise-Aware IR Spectroscopy Signal Processing & Functional Group Identification System

1. Objective

Develop a modular system that:

- Ingests raw IR spectroscopy signals (CSV)
 - Identifies and classifies noise types
 - Applies adaptive filtering using VSSLMS
 - Compares processed signals with a reference IR database
 - Outputs **top compound matches + functional groups**
 - Visualizes all stages via a dashboard
-

2. Problem Statement

Traditional IR signal analysis suffers from:

- Noise contamination (thermal, environmental, detector)
- Static filtering methods (non-adaptive)
- Limited interpretability (no confidence scoring or explainability)

This system solves:

- **Dynamic noise handling**
- **Adaptive filtering**
- **Robust spectral matching**
- **User-facing interpretability**

3. Scope

In Scope

- CSV-based signal ingestion
- Multi-class noise classification
- Adaptive LMS filtering (VSSLMS)
- IR spectral database matching
- Functional group extraction (via peaks)
- Dashboard visualization

Out of Scope

- Real-time hardware sensor integration
 - 2D IR spectroscopy
 - Deep learning-based classification
 - Cloud deployment (optional future scope)
-

4. System Overview

High-Level Workflow

CSV Input → Preprocessing → Noise Classification → Adaptive LMS Filter
→ Compensation Engine → Spectral Matching → Functional Group Detection → Dashboard
Output

This aligns with the 6-module architecture defined in your system design .

5. Functional Requirements

5.1 Signal Ingestion

- Upload CSV file
 - Auto-detect headers
 - Handle missing values (NaN interpolation)
 - Normalize signal amplitude
-

5.2 Preprocessing

- Compute FFT
 - Adaptive cutoff selection
 - Generate:
 - Estimated clean signal (x_{est})
 - Noise estimate
-

5.3 Noise Classification

- Classify into:
 - Non-stationary
 - Detector
 - Environmental
 - Thermal
 - Electronic
 - Apply priority-based decision logic
 - Fallback mechanism for unknown noise
-

5.4 Adaptive Filtering Engine

- Implement VSSLMS algorithm
 - Dynamic step-size (μ) adjustment
 - Stability constraint enforcement
 - Divergence detection and reset
 - Runtime variance monitoring
-

5.5 Compensation Engine

- Select filter parameters (M , μ)
 - Adjust based on signal power
 - Re-evaluate during runtime
 - Post-filter SNR validation
 - Trigger fallback if $\text{SNR} < 0$ dB
-

5.6 Spectral Matching

- Normalize signals
 - Resample database signals
 - Compute:
 - Pearson correlation
 - Spectral Angle Mapper (SAM)
 - Generate top-3 matches with scores
-

5.7 Functional Group Detection

- Extract peaks from filtered signal
 - Map peaks to known IR frequency bands
 - Output detected functional groups
-

5.8 Dashboard (React UI)

Display Components:

- Raw vs Filtered Signal Graph
 - Noise Classification Result
 - Filter Parameters (M , μ)
 - SNR Improvement
 - Top-3 Compound Matches
 - Functional Groups Identified
-

6. Non-Functional Requirements

Performance

- Processing time < 5 seconds for typical signals ($N \leq 10,000$)
- Efficient MATLAB execution

Scalability

- Modular design allows independent upgrades
- Database expansion supported

Reliability

- Fallback mechanisms:
 - Noise classification fallback
 - Filter rerun on poor SNR

Usability

- Simple CSV upload interface
 - Clear visualization of results
-

7. System Architecture

Frontend

- React.js
- Charting library (Chart.js / Recharts)

Backend (Middleware)

- Python (FastAPI preferred)
- Handles:
 - File upload
 - MATLAB execution

- JSON response

Processing Engine

- MATLAB
 - Executes Modules 1–6
-

8. Data Flow

React UI → API → MATLAB Engine → Processing → JSON Output → UI Rendering

9. Data Model

Input

```
{  
  "signal": [float],  
  "sampling_rate": float  
}
```

Output

```
{  
  "noise_type": "Environmental",  
  "snr_before": float,  
  "snr_after": float,  
  "filter_params": {  
    "M": int,  
    "mu": float  
  },  
  "top_matches": [  
    {"name": "Ethanol", "score": 0.87},  
    {"name": "Methanol", "score": 0.82}  
  ],  
  "functional_groups": ["O-H", "C-H"]  
}
```

10. User Stories

Student/User

- Upload a noisy IR signal
- View cleaned signal
- Understand noise type
- Identify compounds and functional groups

Evaluator

- Validate algorithm robustness
 - Inspect filtering performance
 - Compare results with known references
-

11. Success Metrics

- $\geq 85\%$ accuracy in compound matching (on test dataset)
 - Positive SNR improvement in $\geq 90\%$ cases
 - Correct noise classification in $\geq 80\%$ cases
 - Functional group detection accuracy $\geq 75\%$
-

12. Risks & Mitigation

Risk	Mitigation
Misclassification of noise	Tunable thresholds
LMS divergence	Runtime stability constraint
Poor matching accuracy	Combined SAM + Pearson
Integration failure	API abstraction layer

13. Milestones

Phase	Deliverable
Phase 1	MATLAB filtering pipeline
Phase 2	Noise classification + compensation
Phase 3	Database matching
Phase 4	Functional group detection
Phase 5	API integration
Phase 6	React dashboard

14. Future Enhancements

- Real-time sensor integration
 - ML-based classification
 - Cloud deployment
 - Advanced spectral analysis (2D IR)
-

15. Final Positioning

This is not just a filtering tool. It is:

An intelligent IR signal analysis platform combining adaptive DSP, noise intelligence, and chemical interpretation.

